

PROJECT REPORT · 02 OCTOBER 2026

Smart irrigation, informed decisions.

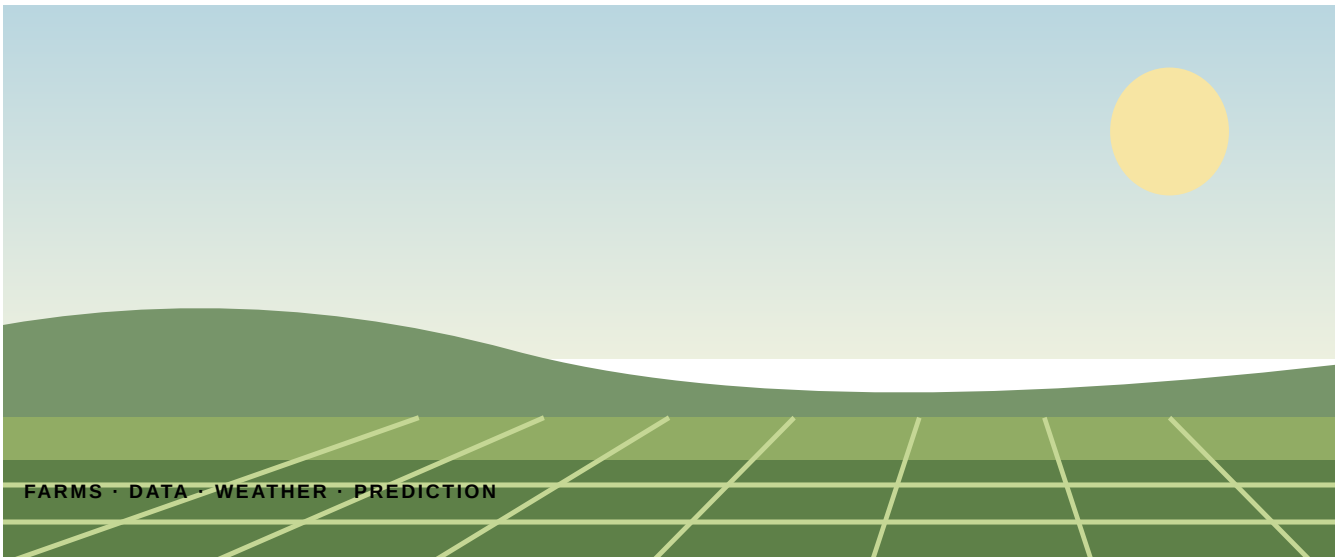
AI-powered farm management for predictive water management
and crop optimization.

FIELD MONITORING

ML RECOMMENDATIONS

VOICE · HINDI · KANNADA

RESPONSIVE PWA



1. Summary

FieldWise is a web-based farm management and irrigation decision-support application. It combines farmer and field records, sensor observations, weather data, and a trained machine learning model to present irrigation recommendations. The application includes a responsive dashboard, regional-language and voice features, alerts, historical views, analytics, and reports.

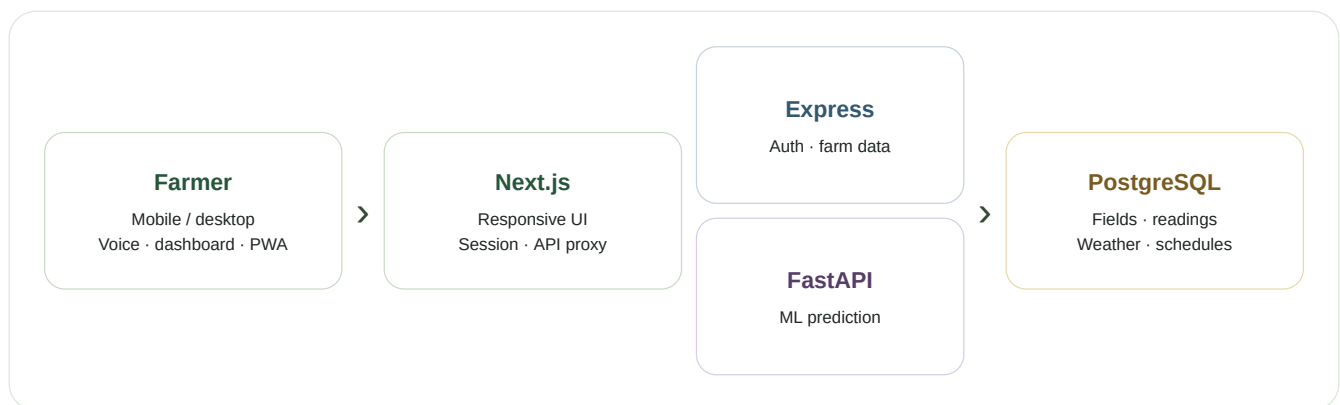
2. Problem and objectives

Irrigation decisions depend on several changing factors, including soil moisture, crop, growth stage, weather, and field conditions. FieldWise brings these inputs together so a farmer can review current conditions and receive a recommendation in one interface. The system also provides a record of readings and recommendations to support follow-up decisions.

Project objectives:

- Provide account-protected access to farmer and field information.
- Collect and display field sensor and weather observations.
- Generate field-specific irrigation predictions and schedules.
- Make core dashboard information usable in English, Hindi, and Kannada, including voice interaction where supported.
- Provide alerts, field history, analytics, exports, and an installable PWA experience.

3. System design



Authenticated service requests use farmer-scoped field access.

Layer	Implementation	Responsibility
User interface	Next.js, React, TypeScript	Responsive dashboard, login, field details, history, analytics, reports, voice and language controls
Application API	Express and Node.js	Authentication, farmer-scoped CRUD, readings, weather, notification preferences, translation and reporting data
Prediction API	FastAPI and scikit-learn	Validates authenticated field requests and runs the saved irrigation prediction pipeline
Persistence	PostgreSQL	Stores farmers, fields, crops, sensor readings, weather, irrigation schedules and notification settings
Simulation	Python sensor simulator	Generates periodic demo readings for active fields
PWA and browser APIs	Service worker, Web Push, Web Speech APIs	App shell caching, optional notifications, speech recognition and text-to-speech

The Express and FastAPI services share a signed session configuration and check field ownership for protected field operations. Optional integrations include OpenWeather, Sarvam, Web Push, Twilio, and SendGrid.

4. Main user flows

1. A farmer creates an account or signs in; the application uses an HTTP-only signed session cookie.
2. The farmer views owned fields, crop details, current sensor readings, weather, and irrigation recommendations.
3. The prediction service applies the saved preprocessing/model artifacts and returns irrigation guidance.
4. Readings and schedules populate the history and analytics views; reports can be exported or printed.
5. A farmer can select a supported language, ask a question by voice or text, and hear a spoken response when browser speech APIs are available.
6. Alerts and external notification channels are opt-in and depend on browser permissions or configured provider credentials.

5. Security and data handling

- Passwords are hashed with Node.js scrypt; sessions are signed and stored in HTTP-only cookies.
- Protected field operations are scoped to the authenticated farmer.
- API credentials for external providers are configured on the backend and should not be committed.
- Authenticated API responses and farmer data are not included in the offline service-worker cache.
- Local LAN HTTPS uses a development CA; each test device must trust that CA. A public deployment requires a certificate issued for its real domain.
- Simulator values are synthetic and are not a substitute for physical sensor readings.

6. Verification summary

The following checks were completed during project review:

- Frontend ESLint and TypeScript checks passed.
- Python compilation across backend, ML, and simulation sources passed.
- The live Next.js HTTPS page, manifest, Express service, FastAPI service, and database health routes responded

successfully.

- Authenticated session and owned-field reads succeeded for field listings, latest sensor data, sensor history, irrigation history, weather history, analytics, and notification preferences.
- Invalid login, unauthenticated access, and malformed request checks returned expected error responses.
- Browser checks covered login/navigation, FieldWise branding, the voice APIs, and a Hindi voice-question response in the available desktop browser.
- The sensor simulator was started and confirmed to generate readings for active fields.

Verification limits

- Successful prediction and irrigation-completion requests write database records. Those write flows were not exercised in the final live check because automatic approval blocked the proposed database mutation.
- Real push, SMS, and email deliveries were not sent. They require credentials, user opt-in, and in some cases verified senders.
- Speech recognition and microphone permissions were not verified on physical mobile devices. Support varies across browser versions and devices.
- A synthetic field output included a negative temperature reading during the simulator run; simulator realism should be reviewed before using its output for demonstrations that imply real measurements.

7. Running the project

See the root [README.md](#) for prerequisites, database initialization, environment setup, HTTPS/LAN instructions, provider configuration, simulator use, API routes, and ML commands.

8. Conclusion

FieldWise provides an integrated application structure for field monitoring and irrigation decision support, with language, voice, notification, and reporting features around the prediction workflow. The current code passes static and read-only runtime checks. Before a formal live demonstration, validate prediction and irrigation-completion writes against disposable test records, exercise voice and PWA behavior on the target mobile browsers, and use clearly labelled synthetic data or verified physical sensor readings.